

PathOn Robotics Take Home Project 1

Dexterous Manipulation in MuJoCo: RUKA Hand + Piper Arm

PathOn Robotics

Project Overview

This take-home project is designed to assess your skills in dexterous manipulation, simulation environment design, and 3D vision-based grasping. Specifically, we want to see how you mount the RUKA dexterous hand onto the AgileX Piper arm as a single integrated robot in MuJoCo, build a realistic tabletop scene with YCB objects, and use a state-of-the-art 3D vision-based dexterous grasping model to perform a pick task in simulation.

Key requirement: The RUKA hand must be mounted to the Piper arm — this is not optional. The entire pipeline (perception, grasp prediction, planning, execution) must operate on the combined Piper + RUKA robot, not on a standalone hand or a different gripper.

Deadline

Due: May 9, 2026 (Saturday), 11:59 PM PST

This project is NOT timed — work at your own pace and submit when ready. You are free to use any AI coding tools like Cursor, Claude Code, Codex, Gemini CLI.

Background

Dexterous manipulation in simulation is a critical capability for modern robotics research. A full pipeline requires three ingredients:

1. A realistic robot model — a dexterous hand mounted on a manipulator arm, with correct kinematics, dynamics, and collision geometry.
2. A realistic scene — objects with physically meaningful properties, placed on a table, with sensors (RGB-D cameras) that replicate what you'd see on a real robot.
3. A perception + grasping pipeline — a 3D-vision-based dexterous grasp predictor that can consume point clouds (or RGB-D) and produce grasp poses feasible for the mounted hand.

In this project you will build all three and demonstrate an end-to-end pick task in MuJoCo.

Robots

- RUKA Hand — 5-finger, 15-DOF underactuated tendon-driven anthropomorphic hand from NYU. Fully open source (CAD, controllers, teleop code).
 - Project: <https://ruka-hand.github.io/>
 - Code: <https://github.com/ruka-hand/RUKA>
 - Paper: <https://arxiv.org/abs/2504.13165>
 - Hardware instructions: <https://ruka.gitbook.io/instructions>
- AgileX Piper — 6-DOF collaborative arm commonly used in lab research.
 - Piper SDK: https://github.com/agilexrobotics/piper_sdk
 - Piper ROS: https://github.com/agilexrobotics/Piper_ros

Objects

Use the YCB object set curated here: - https://github.com/elpis-lab/YCB_Dataset

Pick a subset of objects (at least 5) representative of common grasping benchmarks — e.g., a mug, a banana, a cracker box, a power drill, a mustard bottle.

Grasping approach (reference)

You are free to use either a model-based / learned 3D dexterous grasp approach or a heuristic-based 3D dexterous grasp approach — the choice is yours, as long as you can justify it (accuracy, generality across objects, speed, deployment complexity, how cleanly it maps to RUKA's 15-DOF topology, etc.).

- Model-based / learned dexterous grasping (examples):
 - AnyDexGrasp — <https://github.com/graspnet/AnyDexGrasp> · paper: [arXiv:2502.16420](https://arxiv.org/abs/2502.16420) (“General Dexterous Grasping for Different Hands with Human-level Learning Efficiency”, Fang et al., 2025). Point-cloud input, built on MinkowskiEngine + PointNet2. Ships with implementations for Allegro, DH3, and Inspire hands. Note: license is CC BY-NC 4.0 (non-commercial research use only).
 - UniDexGrasp — <https://github.com/PKU-EPIC/UniDexGrasp>
 - UniDexGrasp++ — <https://github.com/PKU-EPIC/UniDexGrasp2>
 - DexGraspNet — <https://github.com/PKU-EPIC/DexGraspNet>
 - DexGraspNet 2.0 — <https://github.com/PKU-EPIC/DexGraspNet2>
 - Or any other recent cross-embodiment dexterous grasp model (late 2025 / 2026 SOTA is preferred).
- Heuristic / analytical dexterous grasping (examples):
 - Bounding-box / principal-axis heuristics: segment the object point cloud, extract oriented bounding box, wrap the RUKA in a power-grasp template aligned to the shortest axis.
 - Antipodal sampling or force-closure analysis on the object mesh / point cloud to pick finger contact points.
 - Grasp-taxonomy templates (power / precision / pinch / tripod) parameterized by object size and selected by simple geometric rules.
 - Analytical grasp synthesis tools like GraspIt! or the Python successors.

Whichever you choose, cite what you picked and explain why in your report. A well-justified heuristic pipeline that produces reliable grasps on RUKA is better than a misapplied learned model, and vice versa. If you choose a learned model, retargeting the model's output grasps (usually defined for Shadow / Allegro / MANO) to the RUKA's 15-DOF topology is part of the work and will be evaluated.

Goal

Build a MuJoCo simulation environment where:

1. The RUKA hand is mounted to the Piper arm (this is the first and most important step — without this, no subsequent step works correctly). The combined Piper + RUKA robot is placed in front of a table.
2. Multiple YCB objects are placed on the table.
3. A 3D vision-based dexterous grasp model consumes point cloud / RGB-D input from one or more simulated cameras (you choose the configuration — wrist-mounted, scene-mounted, multi-view, whatever you can justify) and proposes grasps for the RUKA hand (mounted on Piper).
4. The best grasp is executed end-to-end on the combined robot: home → pre-grasp → grasp closure → lift → place (or lift-and-hold).
5. The result is recorded and evaluated over a small set of objects.

Tasks

Task 1: Mount the RUKA Hand to the Piper Arm (25%) — REQUIRED FIRST STEP

Objective: Mount the RUKA dexterous hand to the Piper arm and validate the combined robot in MuJoCo. This is the foundational task — every later task depends on it. The project does not accept a standalone RUKA hand or a Piper arm with a different end-effector.

- Obtain or construct MJCF models for both the Piper arm and the RUKA hand. If an official MJCF is not available for RUKA, build one from the CAD/STL assets (the RUKA repo provides CAD via Onshape).
- Mount the RUKA hand to the Piper end-effector. Design a simple wrist adapter / flange mount that attaches the RUKA base to the Piper's tool flange. You may do this purely in the MJCF by aligning frames (no physical CAD needed), but document the transformation (position + orientation offset) clearly.
- Verify the combined Piper + RUKA robot in MuJoCo:
 - Piper's 6 arm joints + RUKA's 15 hand joints all move independently under direct joint commands.
 - Joint limits, masses, and collision meshes are reasonable for both.
 - No self-collision at the home pose, and no interpenetration at the mount interface.
- Deliverables:
 - A single combined MJCF file (Piper arm with RUKA hand mounted) with all meshes.
 - A script that loads the combined model and sweeps each joint through its range.
 - A short README explaining the mount location, the adapter transformation, and any simplifications made.

Task 2: Scene and Sensors (15%)

Objective: Build the tabletop manipulation scene.

- Add a table in front of the robot at a reasonable working height.
- Load at least 5 YCB objects from https://github.com/elpis-lab/YCB_Dataset and place them on the table with sensible physics (mass, friction, restitution).
- Add one or more RGB-D cameras. You are free to configure the camera setup yourself — number of cameras, placement (wrist-mounted, shoulder-mounted, scene / third-person, multi-view), intrinsics (resolution, FOV, depth range), and extrinsics. Justify your choice in the README based on what your chosen grasp model expects as input and what's realistic for the Piper + RUKA setup. Produce point clouds from depth with correct intrinsics/extrinsics.
- Deliverables:
 - Scene MJCF.
 - A script that renders RGB, depth, and a fused point cloud of the scene and saves an example to disk.
 - A short note explaining camera placement choices.

Task 3: 3D Vision-Based Grasp Prediction (25%)

Objective: Propose dexterous grasps for the RUKA hand from 3D observations of the scene.

- Select and justify a grasping approach (see Background) — either a model-based / learned pipeline or a heuristic / analytical pipeline is acceptable, as long as you justify the choice.
- Feed the point cloud from Task 2 into your chosen pipeline.
- Produce a ranked list of candidate grasps (grasp pose + RUKA finger configuration).
- If you chose a learned model whose hand representation differs from RUKA (e.g., Shadow / Allegro / MANO), document your retargeting approach to the RUKA 15-DOF configuration clearly. You may use dex-retargeting or implement your own optimizer.
- If you chose a heuristic pipeline, document the geometric assumptions and how finger poses are generated for RUKA's 15-DOF topology.
- Filter grasps for arm reachability (Piper IK feasibility) and collision-freeness.

- Deliverables:
 - Inference / grasp-generation script.
 - A visualization that overlays the top-K proposed grasps on the scene point cloud.
 - Retargeting or heuristic documentation (whichever applies).

Task 4: Planning and Execution (25%)

Objective: Execute the selected grasp end-to-end.

- Plan and execute a trajectory: home → pre-grasp → approach → close fingers → lift.
- You may use any IK/planning stack (MuJoCo built-ins, PyBullet IK, TRAC-IK, cuRobo, or a simple pseudo-inverse controller).
- Pick at least 3 different objects and report success/failure for each.
- Deliverables:
 - Execution script that takes a grasp proposal and runs the full pick.
 - A per-object result table (object → success/failure + failure mode if any).

Task 5: Evaluation and Demo (10%)

- Produce a 3–5 minute demo video showing:
 - The assembled Piper + RUKA model.
 - The scene with YCB objects.
 - The grasp proposals overlaid on the point cloud.
 - 2–3 successful grasp executions.
- A brief analysis of failure modes and what you'd improve next.

Submission Requirements

Please use MuJoCo for simulation and ROS2 Jazzy is optional (use it if you want arm control / planning to be closer to the real stack; not required).

Deliverables

1. Video demonstration (YouTube / Vimeo / Loom unlisted link) — 3–5 minutes covering the five points above.
2. Code package (ZIP via email — please do NOT upload to a public repo since we use this project for multiple candidates) with:
 - Clear folder structure.
 - README.md with setup, environment (conda / uv / pip), how to run each task, and hardware/software requirements.
 - requirements.txt or environment.yml.
 - Example assets (or download scripts for large assets).
3. Technical report (1–2 pages PDF) covering:
 - Approach — grasp model choice, retargeting method, IK/planning stack.
 - Challenges and solutions.
 - Key learnings and what you would do differently with more time.
 - References cited.

Evaluation Criteria

Category	Weight	What We're Looking For
Functionality	40%	Does the full pipeline run end-to-end? Are at least 2–3 successful grasps demonstrated?
Technical Understanding	30%	Do you understand dexterous hand kinematics, retargeting, 3D vision-based grasping, and sim physics? Are your design choices justified?
Problem-Solving	20%	How did you handle RUKA → model-hand mismatch, unreachable grasps, unstable contacts, or sim-tuning issues?
Communication	10%	Is your code well-organized and your report clear?

Resources

- RUKA project page: <https://ruka-hand.github.io/>
- RUKA code: <https://github.com/ruka-hand/RUKA>
- RUKA paper: <https://arxiv.org/abs/2504.13165>
- Piper SDK: https://github.com/agilexrobotics/piper_sdk
- Piper ROS: https://github.com/agilexrobotics/Piper_ros
- YCB objects: https://github.com/elpis-lab/YCB_Dataset
- Dex retargeting: <https://github.com/dexsuite/dex-retargeting>
- AnyDexGrasp code: <https://github.com/graspnet/AnyDexGrasp>
- AnyDexGrasp paper: <https://arxiv.org/abs/2502.16420>
- AnyDexGrasp project page: <https://www.graspnet.net/anydexgrasp>
- UniDexGrasp: <https://github.com/PKU-EPIC/UniDexGrasp>
- UniDexGrasp++: <https://github.com/PKU-EPIC/UniDexGrasp2>
- DexGraspNet: <https://github.com/PKU-EPIC/DexGraspNet>
- DexGraspNet 2.0: <https://github.com/PKU-EPIC/DexGraspNet2>
- MuJoCo Menagerie: https://github.com/google-deepmind/mujoco_menagerie

Confidentiality

This document and its contents are proprietary to PathOn Robotics. Please treat them as confidential.

- Do not share this PDF publicly — no social media, no public repos, no blog posts, no screenshots of the task description or evaluation criteria.
- Do not share your code publicly — no GitHub / GitLab / any other public hosting. Submit as a ZIP via email only.
- The project description, task structure, evaluation criteria, and any internal assets provided or referenced by PathOn Robotics are the property of PathOn Robotics and may not be redistributed.
- Third-party open-source dependencies (RUKA, Piper, YCB objects, grasping models, MuJoCo, etc.) remain subject to their original licenses.

Violations of confidentiality will disqualify your submission.

Questions?

If you have any questions about the project requirements, technical setup, or clarifications needed, please email:

Rex Zhang, rex@pathon.ai, or chat in the corresponding discord channel on PathOn Robotics Server

Before You Start

Please read this PDF in full — especially the Task breakdown and Evaluation Criteria — before deciding to commit to the project. Only start working if you believe the project is within your ability. If after reading this you feel there is too much missing knowledge to complete it in good faith by the deadline, let Rex know immediately — we can discuss alternatives or stop here, no hard feelings. It is better to flag it early than to spend weeks on something that will not come together.

Important Reminders

- Deadline: May 9, 2026 (Saturday), 11:59 PM PST
- Submission: video link + ZIP file + technical report via email
- Do NOT share this PDF or your code publicly — all materials are proprietary to PathOn Robotics
- We're looking forward to seeing your solution!